

AI Model Selection and Evaluation

Proprietary and open-weight models · local deployment · dataset splitting · evaluation metrics

What this module answers

- What exactly is the difference between proprietary, open-weight and local models?
- How should data be split into train, validation and test sets — and when is cross-validation needed?
- Which metrics should be used for classification, regression, time series and clustering?
- How do we evaluate Generative AI outputs when no single exact answer exists?

Course focus: concrete concepts, decision rules, formulas and examples.

Three different dimensions that are often confused

A model can be classified independently by access, deployment and task.

1. Access to weights

Proprietary
Open-weight

2. Where it runs

Cloud / API
Local / on-prem

3. What it does

Classification
Regression
Generation
Embedding

Important: open-weight $\neq$ local. An open-weight model may run in the cloud; a proprietary model may also be offered in a private hosted environment.

Proprietary models

Weights are controlled by the provider; access is typically through an API or managed service.

● Typical characteristics

- Model weights are not downloadable
- Provider controls training, updates and infrastructure
- Consumption is usually priced by token, request, time or provisioned capacity
- Service terms define data handling, retention, regions and permitted use
- Fast access to strong models without operating the inference stack

● Main trade-offs

- Limited control over model internals and release cadence
- Potential dependency on provider pricing and availability
- Data residency depends on available service regions and contractual options
- Customization usually relies on prompting, RAG, fine-tuning services or adapters exposed by the provider

Open-weight models

The trained parameters are available for download, subject to the model's license.

What is actually open?

- Model weights
- Usually inference code and configuration
- Often tokenizer and model architecture

Not necessarily the training data, training code or full recipe.

What this enables

- Local inference
- Private-cloud deployment
- Quantization
- Fine-tuning / adapters
- Custom serving stack
- Benchmarking under your own controls

Key caveat: the license

- Commercial use may be allowed, limited or conditional
- Redistribution rules vary
- Attribution may be required
- Some licenses add acceptable-use restrictions

Open-weight is not the same as open source

For AI models, 'open' can refer to different artifacts.

Artifact	Open-weight model	Fully open model
Weights	✓ available	✓ available
Inference code	Usually	✓ available
Training code	Sometimes	✓ available
Training dataset	Often not	✓ available / documented
Training recipe / hyperparameters	Often incomplete	✓ reproducible
License	Varies	Open-source / open-data compatible

For procurement and architecture decisions, inspect the exact license and available artifacts — not just the word “open”.

Local models: what does 'local' mean?

Local = inference runs on infrastructure you control

- Developer workstation
- On-premise server or GPU cluster
- Private cloud / dedicated tenant under your control
- Edge device
- Air-gapped environment

Local does NOT automatically mean

- Open source
- Free
- Faster
- More accurate
- Easier to operate

Why run a model locally?

Data control

Prompts and outputs can remain inside the controlled environment.

Residency

Useful where data must remain in a specific country, cloud, network or security zone.

Latency

No public API round trip; useful for high-frequency or edge scenarios.

Cost at scale

For stable high utilization, owned capacity may reduce marginal inference cost.

Availability

Can keep working even when an external provider or Internet connection is unavailable.

Customization

Control over quantization, serving parameters, fine-tuning and model versions.

Trade-off: you become responsible for infrastructure, security hardening, patching, monitoring, scaling and lifecycle management.

Model selection criteria

SELECTION

Quality

Task accuracy, robustness, hallucination rate, domain performance

Latency

Time to first token, tokens/s, p95/p99 response time

Cost

API cost, GPU cost, engineering/operations cost

Data residency

Where prompts, context and logs are processed/stored

Licensing

Commercial use, redistribution, attribution, restrictions

Operability

Observability, versioning, scaling, deployment complexity

Selection is multi-objective: there is rarely a single 'best' model independent of the use case.

Why split a dataset?

We need separate data for learning, tuning and final unbiased estimation.

TRAIN

Fit model parameters

60–80%

VALIDATION

Choose hyperparameters, features, thresholds

10–20%

TEST

Estimate final generalization performance

10–20%

Never use the test set to choose the model, features or thresholds. Otherwise the test set becomes part of the training process.

Holdout splitting: use the right variant

Random split

Use when observations are independent and identically distributed.

Use: Typical tabular ML

Stratified split

Preserve class proportions in each subset.

Use: Classification, especially imbalanced classes

Group split

Keep all rows from the same entity in one subset.

Use: Customers, patients, devices, companies

Time-based split

Train on the past and test on the future.

Use: Forecasting and temporally ordered processes

K-fold cross-validation

Use cross-validation when one holdout split would be too dependent on chance.

1	VALID	TRAIN	TRAIN	TRAIN	TRAIN
2	TRAIN	VALID	TRAIN	TRAIN	TRAIN
3	TRAIN	TRAIN	VALID	TRAIN	TRAIN
4	TRAIN	TRAIN	TRAIN	VALID	TRAIN
5	TRAIN	TRAIN	TRAIN	TRAIN	VALID

Process

- Split data into K folds
- Train K models
- Each fold is validation once
- Average the K validation scores
- Refit the final model on all training data

Common choice: K = 5 or 10

Stratified K-fold

Each fold preserves class proportions.

Classification

Group K-fold

A group appears in only one fold.

Repeated entities / leakage risk

Repeated K-fold

Run K-fold multiple times with different partitions.

More stable estimate

Leave-One-Out

K = number of observations.

Very small datasets; expensive

Nested CV

Inner CV tunes; outer CV estimates performance.

Model selection + unbiased estimate

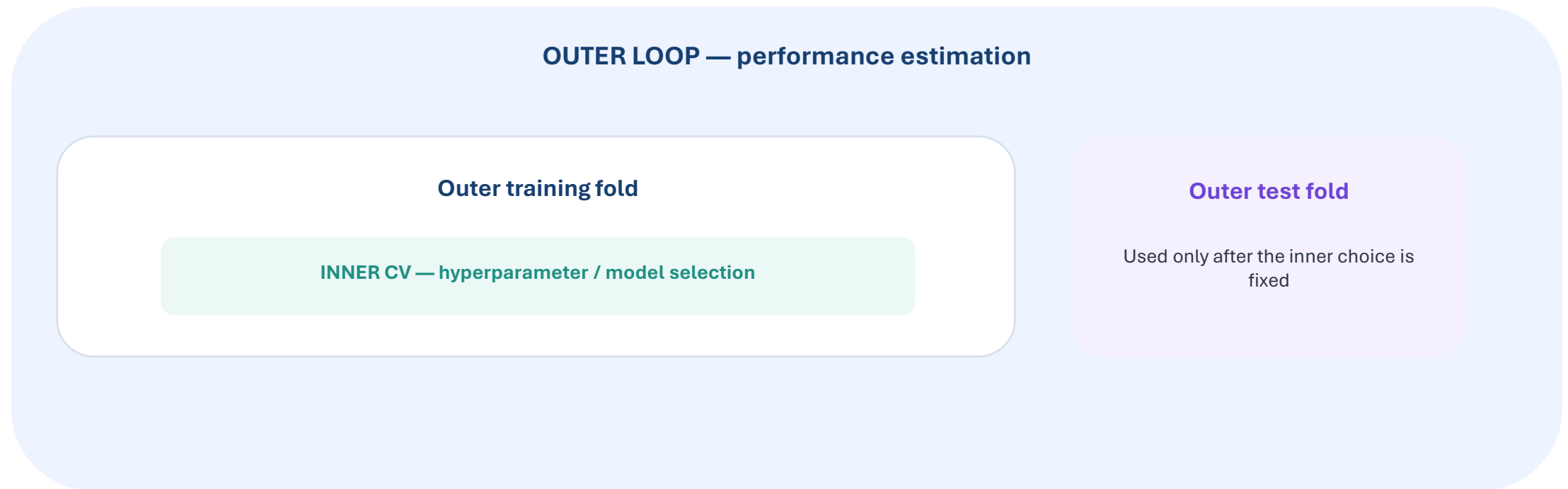
Time-series CV

Validation windows move forward in time.

Forecasting / temporal data

Nested cross-validation: tuning without optimistic bias

Using the same CV loop both to tune and to report performance produces an optimistic estimate.



Average score across the outer test folds = less biased estimate of the whole model-selection procedure.

Binary classification

	Predicted +	Predicted -
Actual +	TP True Positive	FN False Negative
Actual -	FP False Positive	TN True Negative

The business meaning is in the errors

- False Positive: model raises an alarm when the event is absent
- False Negative: model misses an event that is actually present
- The decision threshold changes the balance between FP and FN
- Choose metrics according to the relative cost of these two error types

Classification metrics: what each one measures

METRICS

Accuracy

$$(TP + TN) / N$$

Good only when classes and error costs are reasonably balanced

Precision

$$TP / (TP + FP)$$

Of predicted positives, how many are correct?

Recall / Sensitivity

$$TP / (TP + FN)$$

Of actual positives, how many did we find?

Specificity

$$TN / (TN + FP)$$

Of actual negatives, how many did we reject?

F1-score

$$2 \cdot P \cdot R / (P + R)$$

Harmonic mean of precision and recall

Balanced Accuracy

$$(Recall + Specificity) / 2$$

Useful with class imbalance

Rule of thumb

Rare positive class → inspect precision, recall, F1 and PR-AUC; do not rely on accuracy alone.

ROC-AUC

- Ranks positives above negatives
- Threshold-independent summary
- Can look deceptively strong when the positive class is very rare

Axes: True Positive Rate vs False Positive Rate

PR-AUC

- Summarizes precision–recall trade-off
- More informative for rare positive events
- Sensitive to class prevalence

Axes: Precision vs Recall

AUC does not choose the operating threshold. Select the threshold from business costs, constraints or a target recall/precision.

Regression metrics

METRICS

MAE

$$\text{mean}(|y - \hat{y}|)$$

Same unit as target; easy to explain;
less sensitive to large errors

MSE

$$\text{mean}((y - \hat{y})^2)$$

Strongly penalizes large errors; unit is
squared

RMSE

$$\sqrt{\text{MSE}}$$

Same unit as target; emphasizes
large errors

R²

$$1 - \text{SSE} / \text{SST}$$

Fraction of variance explained
relative to the mean baseline

MAPE

$$\text{mean}(|(y - \hat{y})/y|)$$

Percentage error; problematic when
y is zero or near zero

Median AE

$$\text{median}(|y - \hat{y}|)$$

Robust to large outliers

Regression: choose the metric from the error you care about

Business requirement	Metric	Why
Equal cost per unit of error	MAE	Each extra unit of error contributes linearly
Large errors are especially costly	RMSE / MSE	Squaring makes large residuals dominate
Need relative error across different scales	MAPE / sMAPE	Express error as a percentage; check zero values
Need robust central error	Median AE	A few extreme residuals should not dominate
Need comparison to a simple baseline	R ²	Measures improvement vs predicting the mean

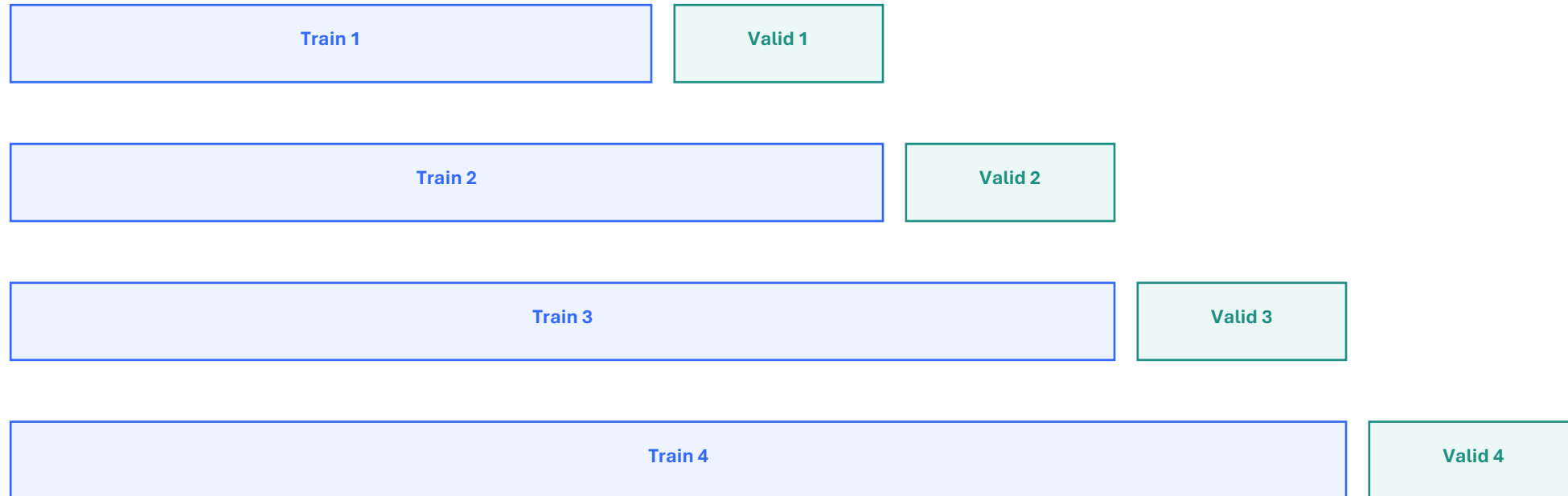
Time series: never shuffle the future into the past

TIME SERIES

Chronology changes both the split strategy and the evaluation method.

PAST

FUTURE



Walk-forward / rolling-origin CV

Time-series forecasting metrics

TIME SERIES

MAE

Average absolute forecast error

Scale-dependent; simple and robust

RMSE

Square-root of mean squared error

Penalizes large misses

MAPE

Mean absolute percentage error

Easy to communicate; unstable near zero

sMAPE

Symmetric percentage error

Reduces asymmetry of MAPE

MASE

MAE scaled by naïve forecast error

Comparable across series; < 1 beats naïve baseline

WAPE

$\Sigma|\text{error}| / \Sigma|\text{actual}|$

Useful aggregate percentage measure

Always compare the model with a naïve baseline such as last value, seasonal naïve or moving average.

Clustering metrics: there is no target variable

Evaluation depends on whether external labels exist.

Internal metrics — no ground truth

- Silhouette score: cohesion vs separation; higher is better
- Davies–Bouldin index: similarity between clusters; lower is better
- Calinski–Harabasz index: between-cluster / within-cluster dispersion; higher is better
- Inertia / WCSS: within-cluster sum of squares; mainly used to compare K in k-means

External metrics — labels are available

- Adjusted Rand Index (ARI)
- Normalized Mutual Information (NMI)
- Purity
- Homogeneity / completeness / V-measure

A numerically 'good' clustering can still be useless. Validate interpretability, stability and business relevance.

Generative AI: evaluation still needs a test set

The expected answer may be a rubric rather than one exact label.

Test case

User input + context + expected constraints

Reference

Optional gold answer / facts / source passages

Rubric

Correctness, completeness, groundedness, style, safety

Scoring

Rule-based metric, human rating or LLM-as-a-judge

Use the same fixed evaluation set to compare prompts, models, RAG versions and system changes.

How it works

- Give the judge the prompt, candidate answer and scoring rubric
- Ask for a structured score and short justification
- Use pairwise comparison when absolute scoring is unstable
- Run on a fixed test set and aggregate results

Controls you still need

- Calibrate against human ratings on a sample
- Check position / verbosity / self-preference bias
- Keep judge model and prompt versioned
- Do not use the same judge as the sole authority for high-risk decisions

1

Define the prediction task and error costs

2

Choose access/deployment: proprietary, open-weight, cloud, local

3

Design a leakage-safe split or cross-validation strategy

4

Pick metrics aligned with the task and business loss

5

Compare against a simple baseline

6

Keep the final test set untouched until the end

7

For GenAI, define a fixed test set + rubric + human calibration